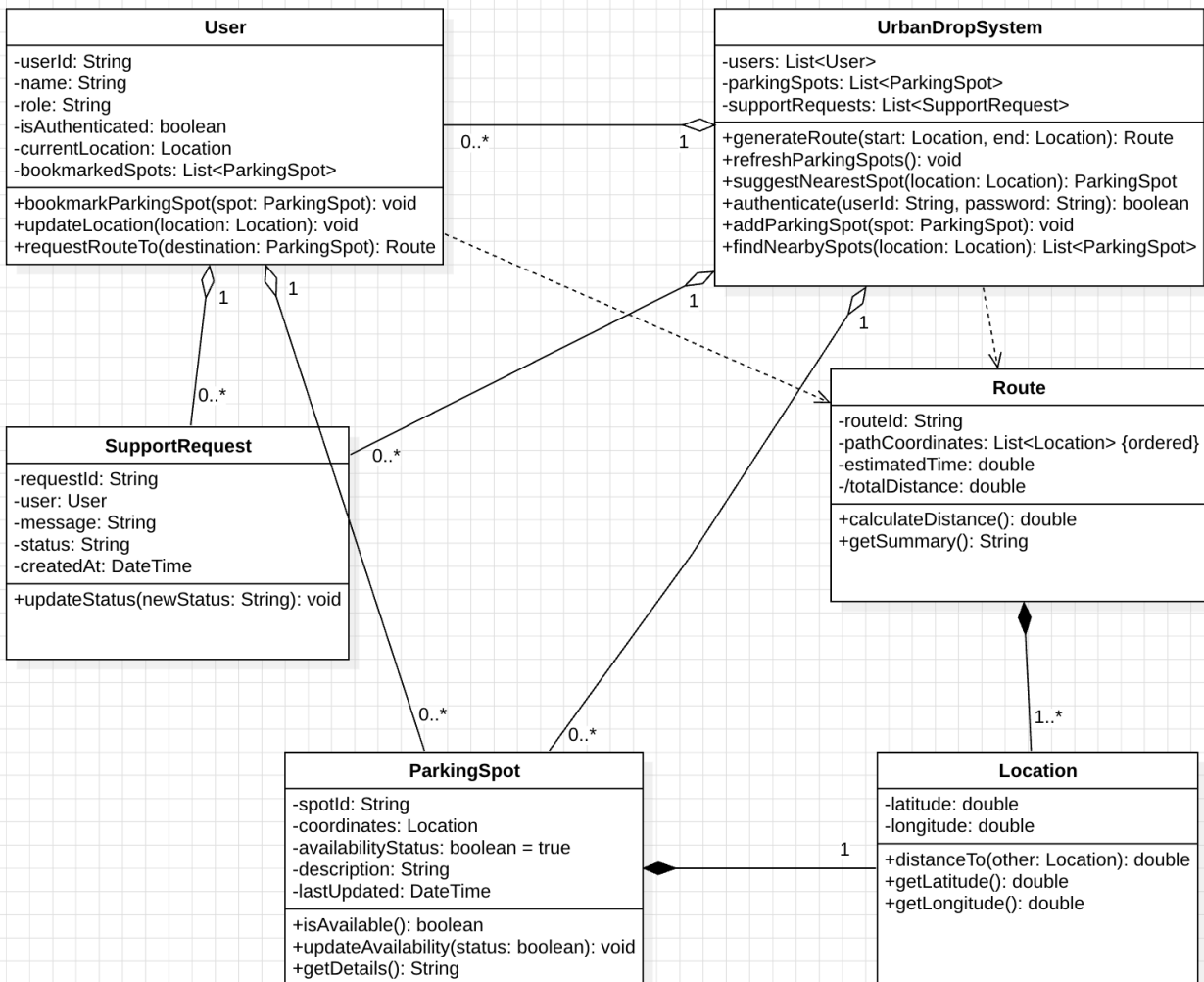


1. UML class diagrams



2. Cohesion (Single Responsibility)

Users are responsible for maintaining user-related information and managing user-specific actions such as bookmarking parking spots and requesting routes. It does not handle route computation or parking spot management logic and just focuses on user behavior and state.

SupportRequest represents a user-generated support inquiry. It encapsulates request-related data, such as message content, status, and creation time, and provides controlled status updates. It does not manage user authentication or system coordination.

ParkingSpot encapsulates parking spot data and availability management. It maintains information about the spot's location, description, and availability status, and provides methods to query or update that status. It does not perform navigation or system-level coordination.

Location represents geographic coordinates and provides distance calculation functionality. By centralizing coordinate-related logic within this class, the system ensures that geographic computations are not scattered across multiple classes.

Route handles route-related data and calculations, including path management, total distance computation, and summary generation. It does not directly interact with user data or system-wide collections.

UrbanDropSystem coordinates system-level operations such as authentication, route generation, and parking spot search. It acts as a mediator between domain objects without embedding their internal business logic.

3. Loose Coupling

User does not compute routes directly. It requests route generation from the **UrbanDropSystem**. This prevents routing logic from being embedded within the **User** class and reduces unnecessary dependencies.

SupportRequest interacts with **User** through object references rather than primitive identifiers, allowing the system to avoid tight coupling to specific ID formats.

ParkingSpot does not perform geographic calculations. All coordinate-related computations are delegated to the **Location** class, ensuring that spatial logic is centralized and not duplicated.

Location hides coordinate calculations and exposes only controlled methods such as `distanceTo()`. Other classes do not directly manipulate latitude and longitude values.

Route does not directly manipulate **ParkingSpot** internals. It operates only on **Location** objects and encapsulates its own path and distance calculations.

UrbanDropSystem coordinates system-level operations but does not expose or directly modify the internal data structures of other classes. It interacts with objects only through their public interfaces, ensuring loose coupling across the system.

4. Hidden Implementation (Encapsulation)

- All member variables are declared private, preventing direct external access.
- External classes must use public methods to interact with objects.
- Internal states such as latitude and longitude are hidden inside the **Location** class.
- The **Location** class exposes only getter methods and distance calculations, preventing direct modification of coordinate values.
- Geographic calculations are encapsulated within the **Location** class rather than being exposed to other classes.

- The totalDistance attribute in the Route class is derived from the pathCoordinates list and is not directly modified by external classes.
- State changes, such as parking availability, can only occur through controlled public methods.

5. Encapsulation of Change

Example 1: Authentication Method Changes

If the authentication mechanism changes (e.g., switching from password-based authentication to token-based authentication), only the UrbanDropSystem class needs to be modified. Other classes, such as User, ParkingSpot, and Route, remain unaffected.

Example 2: Navigation Algorithm Changes

If the route calculation algorithm changes, modifications are confined to the Route and possibly Location classes. Since other classes interact with Route only through its public interface, no further changes are required.

Example 3: Parking Spot Data Format Changes

If additional attributes are added to ParkingSpot (e.g., pricing or time restrictions), only the ParkingSpot class needs to be updated. The overall system architecture remains stable due to encapsulation.